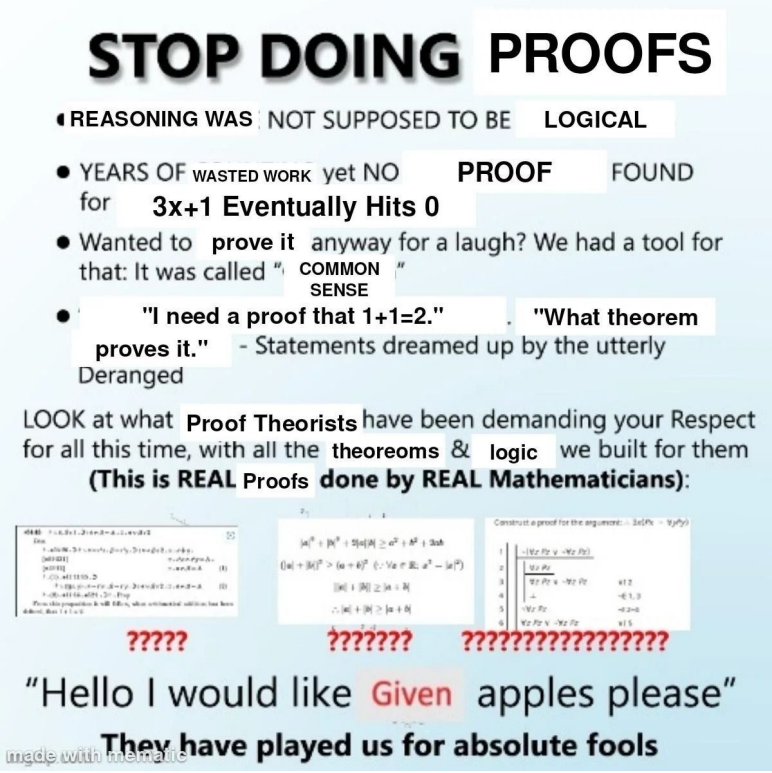
Technische Universität Wien

FORSYTE

Formal Methods in Systems Engineering

course material: slides



aglaia norza

October 15, 2026

norzaglaia@gmail.com
github.com/AglaiaNorza

Contents

1 Satisfiability in Formal Methods **2**
1.1 The SAT Decision Problem 2
1.2 Formula transformation 2
1.3 Resolution 3
1.4 The DPLL Algorithm 5
1.5 Clause-Driven Conflict Learning 7
1.5.1 Implication Graph 7
1.5.2 Clause Learning 9

2 Algebraic Reasoning **12**

3 Program Analysis **13**

4 Model Checking Approaches **14**

CHAPTER 1

Satisfiability in Formal Methods

1.1 | The SAT Decision Problem

> Def. 1 **The SAT decision problem**
>
> Given a propositional formula in conjunctive normal form, is there a truth assignment to the Boolean variables such that it satisfies all clauses?

SAT is an *NP-complete* problem, and it's a very active research area.

1.2 | Formula transformation

We have that

- every formula can be transformed into an **equivalent** CNF formula with, in the worst case, an exponential increase in size
- every formula can be transformed into an **equisatisfiable** formula with a *linear* increase in size (by introducing new variables for subformulae)

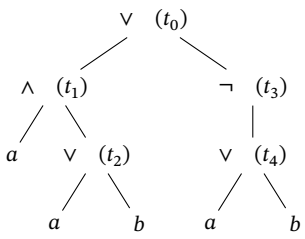
> **Tseitin's transformation**
>
> 1 | introduce a fresh boolean variable for every connective
>
> 2 | encode each equivalence as a set of clauses
>
> 3 | collect all constraints in a big conjunction and enforce the variable representing the entire formula to be true

Example. Consider the following propositional formula:

$$\Phi = (a \wedge (a \vee b)) \vee \neg(a \vee b)$$

First, we break the formula down into an Abstract Syntax Tree (AST) and identify every unique operation, and we then assign a new variable to each (non-atomic) subformula:

$$\begin{aligned} t_0 &\leftrightarrow (t_1 \vee t_3) \\ t_1 &\leftrightarrow (a \wedge t_2) \\ t_2 &\leftrightarrow (a \vee b) \\ t_3 &\leftrightarrow \neg t_4 \\ t_4 &\leftrightarrow (a \vee b) \end{aligned}$$



We translate each of these equivalences into standard CNF clauses via:

- AND $X \leftrightarrow (Y \wedge Z)$:
  $\equiv (\neg X \vee Y) \wedge (\neg X \vee Z) \wedge (X \vee \neg Y \vee \neg Z)$
- OR $X \leftrightarrow (Y \vee Z)$:
  $\equiv (\neg X \vee Y \vee Z) \wedge (X \vee \neg Y) \wedge (X \vee \neg Z)$
- NOT $X \leftrightarrow \neg Y$:
  $\equiv (\neg X \vee \neg Y) \wedge (X \vee Y)$

Finally, to assert that the entire formula Φ is true, we assert the root variable (t_0) as a unit clause, and join it with the CNF encodings of all the subformulae:

$$CNF(\Phi) = t_0 \wedge (t_0 \leftrightarrow (t_1 \vee t_3)) \wedge (t_1 \leftrightarrow (a \wedge t_2)) \wedge (t_2 \leftrightarrow (a \vee b)) \wedge (t_3 \leftrightarrow \neg t_4) \wedge (t_4 \leftrightarrow (a \vee b))$$

We get the fully expanded CNF formula:

$$\begin{aligned} CNF(\Phi) = t_0 \wedge\ & \\ & (\neg t_0 \vee t_1 \vee t_3) \wedge (t_0 \vee \neg t_1) \wedge (t_0 \vee \neg t_3) \wedge \\ & (\neg t_1 \vee a) \wedge (\neg t_1 \vee t_2) \wedge (t_1 \vee \neg a \vee \neg t_2) \wedge \\ & (\neg t_2 \vee a \vee b) \wedge (t_2 \vee \neg a) \wedge (t_2 \vee \neg b) \wedge \\ & (\neg t_3 \vee \neg t_4) \wedge (t_3 \vee t_4) \wedge \\ & (\neg t_4 \vee a \vee b) \wedge (t_4 \vee \neg a) \wedge (t_4 \vee \neg b) \end{aligned}$$

This resulting formula is equisatisfiable to the original Φ.

Plasted-Greenbaum's transformation

Every direction in the Tseitin encoding can be replaced by $\rightarrow$ or $\leftarrow$, where the direction of the implication depends on the *polarity* of the formula

- for every occurrence of a subformula φ, the polarity is $+$ if the number of negations above φ in the syntax tree is even, and $-$ otherwise
- we add $t_i \rightarrow$ if the polarity of t_i is positive, and $\leftarrow t_i$ otherwise
- so, every $\rightarrow$ and $\leftarrow$ can be replaced by at most two clauses

1.3 | Resolution

The theory underlying SAT solving is propositional resolution.

Def. 2 **Resolution**

The resolution rule is:

$$\frac{C \cup \{v\} \qquad D \cup \{\neg v\}}{C \cup D}$$

Which tells us that resolving the clause $C \cup \{v\}$ with the clause $D \cup \{\neg v\}$ results in the resolvant clause $D \cup C$.

We also have a **unit-resolution** rule (or **BCP**, boolean constraint propagation). This rule can

be used on so-called *unit clauses*, clauses with a single literal, which needs to be assigned to $\top$ in a CNF.

$$\frac{C \cup \{\neg v\} \qquad \{v\}}{C} \quad \{I, \neg I\} \cap C = \varnothing$$

> NB!!
>
> This rule is *not refutation-complete*, but it can be applied in time *linear* to the size of the formula.

With the unit-resolution rule, we can do **Unit Propagation**: when a unit resolution produces a unit clause, the unit-resolution step can be repeated with the new clause (until fixpoint).

BCP algorithm

- find a unit clause I
- remove all the clauses in which I occurs
- remove all literals $\neg I$ in clauses
- repeat

Example We want to check the validity of:

$$a \wedge b \rightarrow (a \leftrightarrow b)$$

To check validity, we must check unsatisfiability of the negated formula.

1 | CNF of negated formula:

$$\begin{aligned}
&\neg(a \wedge b \rightarrow (a \leftrightarrow b)) \\
\equiv\ & a \wedge b \wedge \neg(a \leftrightarrow b) \\
\equiv\ & a \wedge b \wedge \neg((\neg a \vee b) \wedge (a \vee \neg b)) \\
\equiv\ & a \wedge b \wedge (\neg(\neg a \vee b) \vee \neg(a \vee \neg b)) \\
\equiv\ & a \wedge b \wedge ((a \wedge \neg b) \vee (\neg a \wedge b)) \\
\equiv\ & a \wedge b \wedge (a \vee \neg a) \wedge (a \vee b) \wedge (\neg b \vee \neg a) \wedge (\neg b \vee b) \\
\equiv\ & a \wedge b \wedge (a \vee b) \wedge (\neg a \vee \neg b)
\end{aligned}$$

2 | Unit resolution on a with $(\neg a \vee \neg b)$

$$a \wedge b \wedge (a \vee b) \wedge (\neg a \vee \neg b) \wedge (\neg b)$$

3 | Unit propagation:

$$b \wedge (\neg b)$$

4 | Unit resolution on b with $(\neg b)$

$$\text{empty clause} \bot$$

The negation is UNSAT, therefore the original formula is valid.

1.4 | The DPLL Algorithm

The DPLL (Davis-Putnam-Logemann-Loveland) algorithm followed the DP (Davis-Putnam) algorithm. Where the first eliminated variables via full resolution, causing an explosion in memory, DPLL branches on variable values, trading memory for execution time (DPLL runs in polynomial space, at the cost of a worst case exponential runtime).

DPLL algorithm

- apply repeated unit resolution and removal of subsumed clauses

  at this point, any literals have been removed and the other clauses have been "cleaned up"
- if $\varnothing \in F$, return UNSAT
- if $F = \varnothing$ (no clauses left), return SAT
- choose a new variable x (using a decision heuristic), and branch on it:
  - add $\{x\}$ and recursively solve
  - add $\{\neg x\}$ and recursively solve
- if one of the branches returns SAT, return SAT
- otherwise, return UNSAT

There is an optional optimisation that can be added (but that could be too expensive compared to simple BCP):

Pure Literal Rule

If a variable appears only with positive polarity (or only negative polarity) throughout F:

- setting that literal to $\top$ cannot falsify any clause
- all clauses containing that pure literal can be eliminated immediately

Kinds of assignments in DPLL:

- **Decided ($\bigwedge$):** arbitrary choice/guess when BCP cannot proceed
- **Propagated (|):** values logically forced by unit resolution (BCP) until $\bot$ or a stall

The main drawback of the DPLL algorithm is its **blindness to found conflicts**:

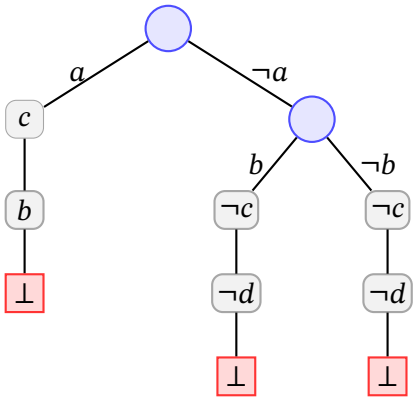
- when a conflict ($\bot$) is encountered, DPLL mechanically undoes only the most recent decision and tries the opposite branch, without analyzing which decisions actually caused the conflict (*chronological backtracking*)
- if a conflict is caused by an independent subset of variables, changing the assignment of an unrelated higher-level variable (like c) forces DPLL to rediscover and prove the exact same conflict all over again
- DPLL does not remember or record why a dead end was reached; it possesses no memory across distinct branches of the search tree

DPLL example: variable choosing Let's consider an example:

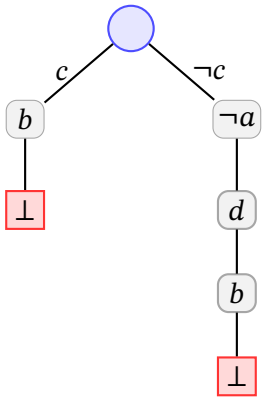
$$\mathcal{F} = (\neg b \vee \neg d) \wedge (b \vee \neg d) \wedge (\neg b \vee \neg c) \wedge (b \vee \neg c) \wedge (\neg a \vee c) \wedge (a \vee c \vee d)$$

Initially, no unit clauses exist, so the solver must decide on an unassigned variable.

The choice of an initial variable can heavily influence the search tree:



Tree 1: first decision on a
(requires 2 decision levels)



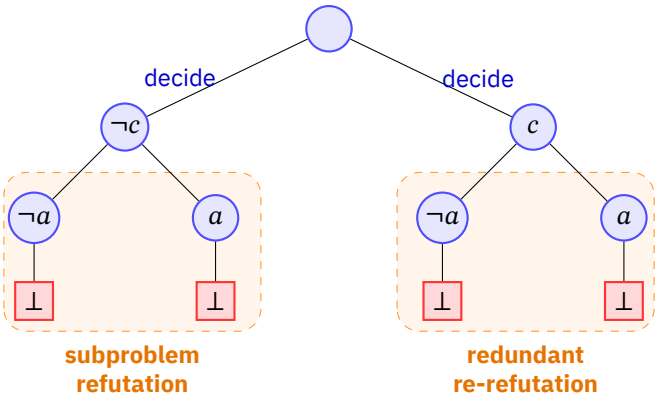
Tree 2: first decision on c
(no secondary decisions needed)

Figure 1.1: DPLL search trees for $\mathcal{F}$. Circles represent decision branch points; rounded rectangles represent literals forced via BCP; and red boxes represent conflict nodes ($\perp$).

DPLL example: blindness to found conflicts Consider the formula:

$$\mathcal{F} = (\neg c \vee a \vee b) \wedge (\neg a \vee \neg b \vee c) \wedge (\neg \mathbf{a} \vee \neg \mathbf{b}) \wedge (\neg \mathbf{a} \vee \mathbf{b}) \wedge (\mathbf{a} \vee \neg \mathbf{b}) \wedge (\mathbf{a} \vee \mathbf{b})$$

Assume the decision heuristic chooses literals in the order: $\neg c \prec c \prec \neg a \prec a \prec \neg b \prec b$.



- under $\neg c$, splitting on $\neg a$ and a yields contradictions because the subformula over a, b is unconditionally unsatisfiable
- chronological backtracking flips $\neg c \rightarrow c$.
- cnder c, DPLL has *no memory* of the previous derivation and blindly duplicates the exact same exploration on $\neg a$ and a

1.5 | Clause-Driven Conflict Learning

Clause-Driven Conflict Learning improves on DPLL by implementing **backjumping**, a technique that helps to recover from bad decisions by skipping redundant conflicts.

To implement backjumping, one needs to recognise, store and update *which decisions* contributed to conflicts. CDCL solvers achieve this through **Conflict Analysis** and **Clause Learning**.

1.5.1 | Implication Graph

An **implication graph** is a labelled DAG $G = (V, E)$ where

- V represents the literals of the current partial assignment (*trail*)

  nodes are labelled with their corresponding literal and decision level ($x = v@d$ where $v \in \{0, 1\}$)
- E represents the reasons of the assignments

  edges are labelled with the reason clause used for propagation
- there is a special node k representing a conflict - it has no outgoing edges, and its incoming edges are:

  $$\{(l, k) | \neg l \in C, C \in F\}$$

  - l is a literal node in the implication graph that has been assigned true
  - $C \in F$ is the clause that caused the conflict (i.e. has been falsified)
  - $\neg l \in C$ is the qualifying condition for the edge: the negation of l is a component of the clause C ($\neg l$ is false, and this falsification contributes to breaking clause C)
  - (l, k) a directed edge is drawn from the node l to the conflict node k, labeled with C to track which clause caused the conflict

> Example
>
> Suppose the formula contains the clause $C = (\neg x_1 \vee x_2 \vee \neg x_3)$.
>
> For C to become a conflicting (falsified) clause, the current decision trail must contain the assignments:
>
> - $x_1 =$ true (node x_1 exists in the graph)
> - $x_2 =$ false (node $\neg x_2$ exists in the graph)
> - $x_3 =$ true (node x_3 exists in the graph)
>
> The nodes responsible for the conflict are x_1, $\neg x_2$, and x_3. Therefore, the incoming edges to the conflict node κ will originate from x_1, $\neg x_2$, and x_3, representing the assignments that caused C to fail.

Implication Graph Example. Let the formula F be:

$$F = C_1 \wedge C_2 \wedge C_3 = (\neg a \vee \neg b \vee c) \wedge (\neg b \vee \neg c) \wedge (\neg a \vee b)$$

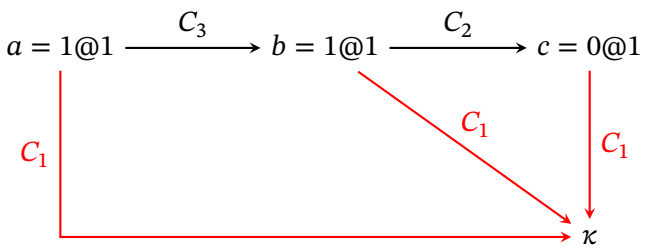
Suppose our first decision is to assign a to true (at decision level 1). This triggers Boolean Constraint Propagation (BCP) as follows:

1 | $a = 1$ forces $b = 1$ to satisfy C_3

2 | $b = 1$ forces $c = 0$ to satisfy C_2

3 | now $a = 1$, $b = 1$, and $c = 0$, which falsifies clause C_1

This propagation and resulting conflict can be visualized in the implication graph below:



Simple implementation details

In modern CDCL solvers, the implication graph is not stored as an explicit graph data structure. Instead, it is implicitly maintained via the assignment trail (stack of currently assigned variables).

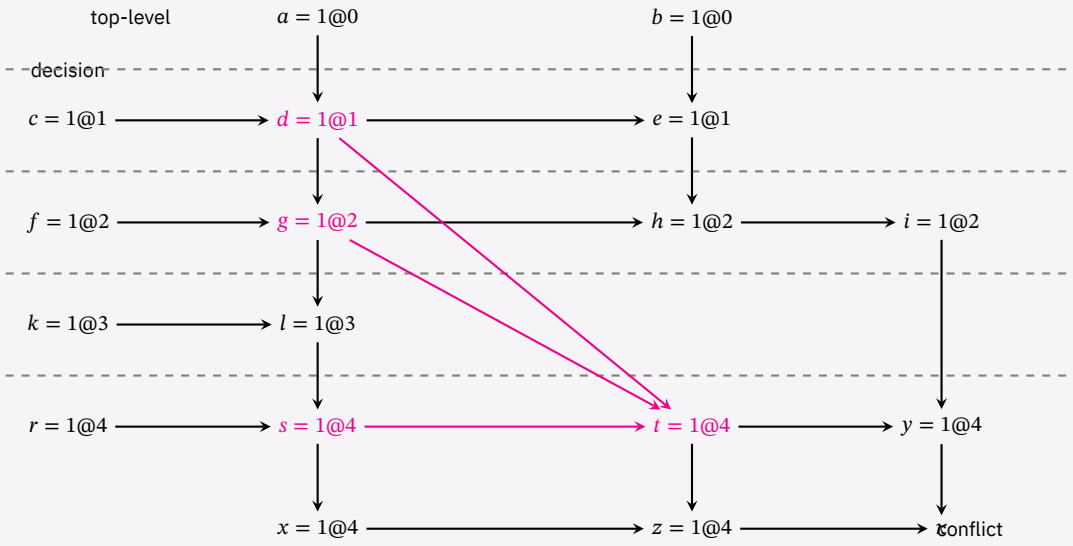
For each assigned variable, we store a tuple `(Value, Decision Level, Reason Pointer)`:

- **reason pointer (antecedent):** pointer to the clause that forced this assignment

  storing this pointer allows the solver to instantly trace backward from a conflict to compute a learned clause via resolution, without re-scanning the clause database

- **decision variables:** variables chosen by the decision heuristic (guesses) are not forced by any clause. Their reason pointer is set to `null` (so,during conflict analysis, a `null` pointer signifies a root node in the implication graph)

Exercise: how to read the graph



$$d \wedge g \wedge s \rightarrow t \quad \equiv \quad (\bar{d} \vee \bar{g} \vee \bar{s} \vee t)$$

Implication graphs allow us to trace the causes of variable assignments and conflicts.

1 | **Finding the cause of an assignment (the antecedent):**
- locate the node n whose assignment you want to analyze
- trace its incoming edges (look at the arrows pointing into n); let's say they come from nodes a, b, and c
- reconstruct the underlying clause: the edges imply that $a \wedge b \wedge c \rightarrow n$; converting this implication into standard CNF format gives us: $\neg a \vee \neg b \vee \neg c \vee n$

2 | **Finding the conflicting clause for a conflict node (κ):**
- the same logic applies, but the implication has no consequent (it implies a contradiction/false)
- trace the incoming edges to κ (e.g., from nodes x and y)
- simply negate these parent nodes and connect them with disjunctions; the clause that caused the conflict is $\neg x \vee \neg y$

1.5.2 | Clause Learning

To help us decide where to backjump to, and to learn more from a found conflict, we can add **clause learning** to our algorithm.

Whenever we encounter a conflict clause, we can add it to the formula: the literals contributing to a conflict form a partial assignment, which is just a conjunction of literals, whose negation is a clause that is *implied by the original CNF*. Therefore, adding this clause prevents the partial assignment from happening again during the search.

Finding cuts in the graph

A visual way to see this is to imagine it as finding cuts in the implication graph. We can draw a **cut** through the implication graph that separates the decision variables (roots) from the conflict node κ (leaf).

However, not all cuts are useful. There are two extremes we want to avoid:

- the decision cut (closest to roots)
  - results in a disjunction of all the negated decisions that led to the conflict
  - it is usually very long and provides almost no new information: it essentially just tells the solver, "don't make this exact combination of guesses again," which does not help prune the remaining search space efficiently
- the conflict cut (closest to leaf):
  - results in the exact clause that was just falsified to cause the conflict
  - gives no new information: the solver already has this clause in its original formula database

Because the decision cut is too broad (weak) and the conflict cut is redundant, modern CDCL solvers look for a cut in the middle of the graph. The goal is to find a cut that produces a **short** clause (because shorter clauses restrict the search space more heavily) and contains **novel information** not explicitly written in the original formula.

Unique Implication Points (UIPs) and Asserting Clauses

To find the optimal cut in an implication graph, modern solvers rely on finding a ”bottleneck” that forces the conflict. This bottleneck is called a Unique Implication Point.

Def. 3 **Unique Implication Point (UIP)**

A node in the implication graph at the *current* decision level d is a UIP if *every path* from the decision variable at level d to the conflict node κ must pass through it.

- in graph theory, a UIP is known as a **dominator** of the conflict node with respect to the decision node
- (it represents a single, unavoidable consequence at the current decision level that triggered the conflict; if you remove the UIP, the conflict cannot happen)

> Nota Bene
>
> The decision variable itself is *always* trivially a UIP, because all paths at the current level originate from it.

Because there can be multiple UIPs in a single implication graph (e.g., the decision variable, and several intermediate forced variables), we must choose for our cut.

The best one to choose is almost always the **First UIP**, i.e. the one *closest to the conflict node* k.

> Why the First UIP?
>
> - *Locality:* it captures the most recent, localized reasons for the conflict, making the resulting learned clause highly relevant to the current search space
> - *Efficiency:* the algorithm to find the 1UIP works backward from the conflict; stopping at the *first* UIP we encounter is computationally faster than resolving all the way back to the decision variable

When we perform resolution backward from the conflict and stop at the First UIP, the resulting learned clause is called an **Asserting Clause**.

Def. 4 **Asserting Clause**

A conflict clause that contains *exactly one literal from the current decision level* (which is the negation of the UIP node). All other literals in the clause come from earlier, lower decision levels.

This structural property of the asserting clause makes non-chronological backjumping work perfectly:

- when we learn an asserting clause, we look at the decision levels of its literals and backjump to the **second-highest decision level** present in the clause
- because the clause only had *one* literal from the highest level, backjumping clears that assignment
- upon arriving at the second-highest level, **every literal in the learned clause evaluates to False, except for one** (the UIP literal, which is now unassigned)

- this immediately triggers Boolean Constraint Propagation; the learned clause *asserts* the forced assignment of the UIP literal, driving the solver forward without needing a new blind guess

CHAPTER 2

Algebraic Reasoning

CHAPTER 3

Program Analysis

CHAPTER 4

Model Checking Approaches